

| Tauriで考える「その処理どっちでやる？」

——認証とセッション管理から見る React と Rust の境界線

フロントエンドカンファレンス関西 2026

田中博悠 / tanahiro2010

自己紹介

- 田中博悠 / tanahiro2010
- 三田学園高等学校 1年生
- プログラミングが好き
- 普段はWeb畑在住
- React / Next.js / Rails はかなり使う
- 最近は Tauri にハマっている

今日の話

OAuthの実装解説ではありません

Google OAuthの実装方法なら、検索すれば出てきます
PKCEの仕組みなら、公式ドキュメントがあります

では何を話すのか

今日の話

「その処理どっちでやる？」
という話です

Tauriを書く

React



invoke



Rust



OS

すると悩めます

- OAuthはどっち？
- セッション管理はどっち？
- ファイル操作はどっち？
- バックグラウンド処理はどっち？

今日のテーマ

React と Rust

どちらに処理を置くべきか？

実は

正解はありません

なぜなら

アプリによって

守るべきものが違うから

だから今日は

実際に私が開発したアプリを題材に
考えてみます

きっかけ

ある日

学校から相談を受けました

相談内容

「Google Classroomを見ない生徒が多い」

最初は

「それは見ればいいのでは？」
と思いました

でも話を聞くと

意外と理由がありました

学校支給端末

- Surface
- Windows
- ブラウザ中心

スマホとの違い

スマホ

→ 通知が来る

Surface

→ 通知がほぼ来ない

結果

Classroomを開かない



課題に気付かない



提出が遅れる

そこで

通知を集約するアプリを作ることになりました

最初は簡単だと思っていました

Google APIを叩いて

表示するだけ

でも

開発を始めてすぐに気付きます

あれ？

これ

どこで処理するんだ？

依頼内容

「高校生がGoogle Classroomを見ない」

理由

Surfaceにはスマホみたいな通知がない



Classroomを開かない



課題を見逃す

そこで作ったもの

未読通知集約アプリ

機能

- Gmail取得
- Classroom取得
- 通知表示
- ログイン維持
- インストール時スタートアップ登録
- 定期通知ポーリング

開発中の疑問

その処理

どっちでやる？

メリットに関して

React ?

- 実装が楽
- 速く作れる
- 慣れている

Rust ?

- OSに近い
- パフォーマンスが高い
- 権限管理しやすい

デメリットに関して

React ?

- 詳細な操作が苦手
- Rustに比べるとパフォーマンスが低い

Rust ?

- 学習コスト / 実装コストが高い
- 型が強力な反面Reactとの橋渡しに苦勞する
- コンパイルが遅い

まず認証

Google OAuthを実装したい
ということで3つの案を

案1 localhost callback

特徴

- サーバー不要
- OAuthでよく使われる
- アプリ内で待受する

良い点

- シンプル
- 導入事例が多い

気になった点

- Rust側の実装が必要

案2

PKCE

デスクトップアプリ向け

案3

中継サーバー

Web側に認証を任せる

それぞれの案の比較

項目	localhost	PKCE	中継
実装難易度	○	△	△
運用	◎	◎	×
拡張性	△	○	◎

◎ とても高い

○ 高い

△ 普通

× 低い

結局何を選んだ？

実はそこが本題ではない

本当に悩んだこと

認証情報

どこに置く？

選択肢

- localStorage
- sessionStorage
- SQLite
- Keychain
- Rust側管理

localStorage / sessionStorageでよくない？

最初はそう思った

理由

- 実装が簡単
- Reactだけで完結
- すぐ動く
- 学内利用だから

でも気付いた

扱うデータが強い

このアプリが読めるもの

- Gmail
- Classroom
- ユーザー情報

しかし、ユーザー入力はない

強いて言うならボタンクリックだけ



でも、安全？

ふと頭に浮かんだ、その疑問。

考えてみたら、そうでもなかった

ユーザー入力はなくても、外部入力は存在するから

外部入力の例

Google系統APIから取得した

- メール本文
- Classroom投稿
- コメント
- 添付ファイル名

などが

重要な気付き

ユーザー入力がなくとも安全とは限らない

もし何か起きたら？

認証情報流出



Googleアカウント連携悪用

僕のソフト的にReadonlyスコープしかないとはいえ、
プライベートな情報を扱っている可能性はたくさんある。

もし何か起きたら？

ファイルアクセス悪用



端末への影響

それに、OAuthだけの話ではない

悩んだのはさっき言った

- 認証
- セッション

に加え、

- 定期ポーリング
- スタートアップ登録

だった

考え方が変わった

実装場所ではなく

被害規模を見る

Reactに置いてよいもの

- UI状態 / 表示設定
- テーマ
- ばれても大丈夫な情報（アイコン画像とか）
- 簡単なフックなど

慎重になるもの

- OAuthトークン
- セッション
- ファイル操作
- OSに近い操作

ReactとRustの境界線

React



invoke



Rust



OS

判断基準

この機能が突破されたら
何が起きる？

例えば、商用サービスなら？

より厳しく考えてみる

内部利用ツールなら？

許容できるリスクもある

大事だったこと

完璧な設計

ではなく

適切な設計

学んだこと

Reactは便利

Rustは安全

どちらかではない

両方使う

今日のまとめ

- OAuthは入口
- 本質は責任分界点
- 被害規模で考える

Takeaway

「その処理どっちでやる？」



「壊れたら何が起きる？」

で考える

 **ご清聴ありがとうございました**

質問歓迎です